

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

Кафедра Информационных систем и технологий

ОТЧЁТ ПО ПРАКТИКЕ
ЗАЩИЩЁН С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

Зав. каф., д.т.н., доцент

должность, уч. степень, звание

С.В. Мичурин

инициалы, фамилия

ОТЧЁТ ПО ПРАКТИКЕ

вид практики производственная

тип практики технологическая (проектно-технологическая)

на тему индивидуального задания Разработка пользовательского веб-интерфейса и
сервиса взаимодействия через мессенджер для системы интеллектуальной диспетчеризации
логистических рейсов на основе искусственного интеллекта

выполнен Томчуком Григорием Сергеевичем

фамилия, имя, отчество обучающегося в творительном падеже

по направлению подготовки

09.03.02

код

Информационные системы и технологии

наименование направления

направленности

наименование направления

03

код

Информационные технологии в дизайне

наименование направленности

наименование направленности

Обучающийся группы № 4326

номер

24.07.2026

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт–Петербург 2026

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

на прохождение производственной технологической (проектно-технологической) практики обучающегося направления подготовки 09.03.02 Информационные системы и технологии

Фамилия, имя, отчество обучающегося: Томчук Григорий Сергеевич

Группа: 4326

1 Тема индивидуального задания:

Разработка пользовательского Веб-интерфейса и сервиса взаимодействия через мессенджер для системы интеллектуальной диспетчеризации логистических рейсов на основе искусственного интеллекта.

2 Исходные данные:

Программный интерфейс системы заявок логистической компании, среда Twin24.

Технологии: React, TypeScript, TanStack Router WebSocket Tailwind, Max Bot API, Docker.

>70% рейсов без вмешательства оператора, < 30 минут ошибка расчета ETA.

3 Содержание отчетной документации:

3.1. индивидуальное задание;

3.2. отчёт, включающий в себя:

- титульный лист;
- материалы о выполнении индивидуального задания (отчет)/
- выводы по результатам практики;
- список использованных источников.

3.3. отзыв руководителя от профильной организации (при прохождении практики в профильной организации).

4 Срок представления отчета на кафедру: «25» июля 2026 г.

Руководитель практики

Зав. каф., д.т.н., доцент,
(должность, уч. степень, звание)

 29.06.2026
(подпись, дата)

С.В. Мичурин
(инициалы, фамилия)

СОГЛАСОВАНО

Руководитель практики от профильной организации

ген. директор
(должность, уч. степень, звание)

29.06.2026
(ПОДПИСЬ, ДАТА)

Д.А. Яснов
(инициалы, фамилия)

Задание принял к исполнению обучающийся

_____ 29.06.2026
(дата)

(подпись)

Томчук Г.С.
(инициалы, фамилия)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Анализ предметной области и постановка задачи	6
1.1 Особенности логистической диспетчеризации	6
1.2 Концепция системы	7
1.3 Требования к пользовательскому интерфейсу	9
1.4 Постановка задачи индивидуального задания	11
2 Архитектура и технологические решения системы	13
2.1 Общая архитектура системы	13
2.2 Технологический стек и организация разработки	14
2.3 Архитектура frontend-компонента	17
2.4 Архитектура сервиса МАХ-бота	19
3 Разработка пользовательского веб-интерфейса и сервиса МАХ-бота	21
3.1 Разработка структуры и основных компонентов пользовательского интерфейса	21
3.2 Реализация страницы обзора и интерфейса работы с инцидентами	23
3.3 Реализация страниц мониторинга и администрирования	26
3.4 Разработка сервиса взаимодействия через мессенджер МАХ	28
4 Результаты разработки и перспективы дальнейшего развития системы	30
4.1 Реализованные результаты разработки	30
4.2 Взаимодействие разработанных компонентов с общей системой	31
4.3 Текущий статус проекта и дальнейшее развитие	32
ЗАКЛЮЧЕНИЕ	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37

ВВЕДЕНИЕ

Современные логистические процессы требуют постоянного контроля большого количества рейсов, транспортных средств и участников перевозки. В ходе выполнения рейса необходимо отслеживать его текущее состояние, контролировать расчетное время прибытия, выявлять возможные отклонения от плановых показателей и поддерживать коммуникацию с водителем. При традиционном подходе значительная часть этих операций выполняется логистом вручную, что увеличивает нагрузку на специалиста и затрудняет оперативную обработку большого количества одновременно выполняемых рейсов.

Одним из направлений решения данной проблемы является автоматизация диспетчерских процессов с использованием технологий искусственного интеллекта. В рамках проекта «ИИ-диспетчер рейсов MKTech» разрабатывается система интеллектуальной диспетчеризации логистических рейсов. Ее задача заключается в автоматизированном контроле состояния перевозок, взаимодействии с водителями и выявлении ситуаций, требующих вмешательства логиста.

В системе используется ИИ-диспетчер на основе GigaChat от Сбера. Он должен анализировать информацию о рейсах и коммуникациях, выявлять отклонения, оценивать возможные риски и выполнять действия в рамках заданных правил. При возникновении нестандартной или критической ситуации система формирует инцидент и передает его на рассмотрение логисту. Такой подход позволяет перейти от постоянного ручного контроля всех рейсов к управлению исключительными ситуациями.

Особое значение в проекте имеет пользовательский веб-интерфейс. Его основная задача заключается не в полноценном ручном управлении каждым рейсом, а в предоставлении логисту актуальной информации о возникающих инцидентах и работе ИИ-диспетчера. Пользователь должен иметь возможность быстро определить критичность ситуации, изучить историю событий и коммуникаций, понять причины принятого ИИ-решения и при

необходимости вмешаться в работу автоматизированной системы.

В рамках индивидуального задания выполнялась разработка пользовательского веб-интерфейса и сервиса взаимодействия через мессенджер MAX для системы интеллектуальной диспетчеризации логистических рейсов на основе искусственного интеллекта. Работа выполнялась в роли frontend-разработчика и разработчика бота в мессенджере MAX.

Целью работы являлась разработка frontend-компонента системы, обеспечивающего мониторинг инцидентов и рейсов, отображение событий и коммуникаций, визуализацию решений ИИ-диспетчера и предоставление логисту инструментов для управления ситуациями, требующими ручного вмешательства.

Для достижения поставленной цели были определены следующие задачи: проанализировать предметную область и требования к системе; определить архитектуру и технологический стек frontend-компонента; разработать пользовательский интерфейс для мониторинга инцидентов и рейсов; реализовать отображение событий, коммуникаций и решений ИИ; разработать интерфейсы мониторинга и администрирования; обеспечить взаимодействие frontend с backend через API и WebSocket; определить архитектуру сервиса взаимодействия с мессенджером MAX и реализовать его базовую функциональность.

В результате работы были разработаны основные пользовательские интерфейсы системы, включающие общий интерфейс приложения, страницу обзора, страницу мониторинга и административную панель. Также была сформирована архитектура MAX-бота, подключен «MAX Bot API» и реализованы базовые сценарии взаимодействия с пользователем.

В настоящем отчете рассматриваются предметная область проекта, архитектура и технологические решения системы, процесс разработки пользовательского веб-интерфейса и сервиса MAX-бота, а также достигнутые результаты и дальнейшие направления развития проекта.

1 Анализ предметной области и постановка задачи

1.1 Особенности логистической диспетчеризации

Логистическая диспетчеризация представляет собой комплекс процессов, связанных с организацией и контролем грузовых перевозок. В ходе выполнения рейса необходимо отслеживать состояние транспортного средства, контролировать соблюдение плановых сроков доставки, анализировать изменения расчетного времени прибытия и поддерживать коммуникацию с водителем. При этом информация о перевозках может поступать из различных внешних систем, а взаимодействие с водителем может осуществляться через несколько каналов связи.

Традиционно значительная часть указанных операций выполняется логистом вручную. Специалисту необходимо одновременно контролировать множество рейсов, отслеживать изменения их состояния, связываться с водителями, уточнять причины задержек и оценивать необходимость дальнейших действий. При увеличении количества перевозок такой подход приводит к росту нагрузки на логиста и повышает риск того, что критическое отклонение будет обнаружено с задержкой.

Дополнительную сложность создает необходимость работы с различными источниками данных. В рамках проекта предусматривается взаимодействие с данными, поступающими из клиентских TMS (Transportation Management System, система управления перевозками) и других внешних систем. В исходной концепции проекта рассматривается подключение нескольких источников информации о перевозках, включая системы, используемые различными клиентами. Это означает, что для эффективной диспетчеризации необходимо не только получать данные о рейсах, но и объединять их в едином процессе обработки.

В процессе выполнения рейса может изменяться его расчетное время прибытия. На ETA (Estimated Time of Arrival, расчетное время прибытия) влияют дорожная ситуация, задержки при погрузке и выгрузке, изменение маршрута, технические проблемы и другие факторы. Если изменение ETA

приводит к повышению вероятности нарушения согласованного срока доставки (SLA, Service Level Agreement, соглашение об уровне обслуживания), возникает необходимость в дополнительном анализе ситуации и, возможно, коммуникации с водителем или клиентом.

В рамках разрабатываемой системы автоматизация направлена на сокращение количества рутинных операций, выполняемых логистом вручную. Система должна самостоятельно анализировать поступающие данные, выявлять отклонения и передавать специалисту только ситуации, требующие его внимания. Такой подход позволяет перейти от постоянного контроля каждого рейса к управлению исключениями.

На рисунке 1 представлена схема традиционного процесса диспетчеризации и его автоматизации в рамках разрабатываемой системы.



Рисунок 1 — Сравнение традиционного процесса диспетчеризации и модели работы ИИ-диспетчера

1.2 Концепция системы

Проект представляет собой систему интеллектуальной диспетчеризации логистических рейсов. Ее основная задача заключается в автоматизации процессов наблюдения за перевозками, взаимодействия с водителями и обработки отклонений от нормального сценария выполнения рейса.

Ключевым компонентом системы является ИИ-диспетчер на основе GigaChat от Сбера. Он должен анализировать информацию о рейсе и события, связанные с его выполнением, взаимодействовать с водителем через доступные каналы коммуникации и принимать решения в рамках заданных правил. В качестве каналов взаимодействия рассматриваются голосовые звонки, сообщения в мессенджере «MAX» и SMS.

Общая логика работы системы строится следующим образом. Система получает данные о текущем состоянии рейса из доступных источников, анализирует их и определяет, соответствует ли ситуация нормальному сценарию выполнения перевозки. При обнаружении отклонения ИИ-компонент может инициировать дополнительные действия, например взаимодействие с водителем или формирование уведомления.

Если ситуация может быть обработана автоматически, ИИ-диспетчер продолжает выполнение сценария без непосредственного участия логиста. Если же обнаружено критическое отклонение или система недостаточно уверена в принятом решении, формируется инцидент. Он передается в интерфейс логиста для анализа и возможной эскалации.

Таким образом, ключевой принцип системы заключается в использовании модели управления исключениями. Логист не обязан постоянно вручную контролировать каждый рейс. Вместо этого система самостоятельно наблюдает за перевозками и обращает внимание специалиста на ситуации, которые требуют вмешательства.

В рамках проекта важную роль играет объяснимость решений искусственного интеллекта. ИИ-диспетчер должен не только сформировать результат, но и предоставить информацию, позволяющую понять логику принятого решения. В пользовательском интерфейсе предусматривается отображение распознанного намерения, извлеченных сущностей, уровня уверенности ИИ, активированных правил и источника информации, использованного при принятии решения.

Для анализа работы ИИ-компонентов применяются LangChain и интеграция langchain-gigachat, а для наблюдаемости и трассировки ИИ-процессов используется Langfuse. Это позволяет отслеживать выполнение ИИ-операций и использовать полученную информацию при анализе качества работы системы.

Важной частью общей концепции является сохранение возможности

человеческого контроля. ИИ-диспетчер автоматизирует типовые операции, но логист сохраняет возможность вмешаться в процесс, изучить контекст ситуации и принять решение самостоятельно. Именно поэтому пользовательский интерфейс является не второстепенным отображением данных, а важным компонентом системы.

На рисунке 2 представлена концептуальная схема работы ИИ-диспетчера: получение данных о рейсе, анализ ситуации, взаимодействие с водителем, принятие ИИ-решения и формирование инцидента при необходимости вмешательства логиста.

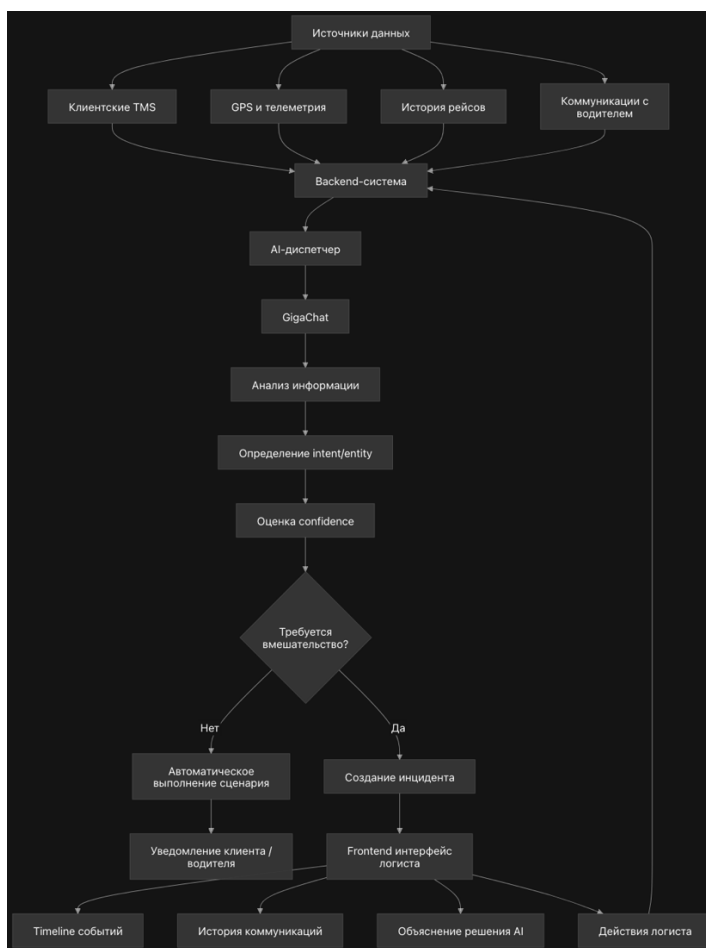


Рисунок 2 — Концептуальная схема работы ИИ-диспетчера и взаимодействия компонентов системы

1.3 Требования к пользовательскому интерфейсу

Основным пользователем разрабатываемого веб-интерфейса является логист-наблюдатель. В отличие от классической операционной CRM-системы, интерфейс не должен заставлять пользователя вручную

обрабатывать каждый рейс. Его основная задача заключается в предоставлении информации об исключениях, контроле работы ИИ-диспетчера и обеспечении быстрых действий при возникновении проблем.

В связи с этим основным объектом интерфейса являются инциденты. Пользователь должен иметь возможность быстро определить количество активных проблем, их критичность и текущий статус обработки. В интерфейсе отображаются открытые и критические инциденты, рейсы с риском нарушения SLA, эскалированные рейсы (взятые под ручной контроль), средняя уверенность ИИ и среднее отклонение ETA.

Для анализа конкретной ситуации логист должен иметь возможность открыть подробную информацию об инциденте. В детализации необходимо объединить данные о рейсе, событиях, коммуникациях с водителем и действиях ИИ. Это позволяет пользователю восстановить последовательность развития ситуации и понять причины возникновения проблемы.

Для отображения истории используется событийная модель. В цепочке должны отображаться значимые события, связанные с рейсом и инцидентом: изменения состояния, изменения ETA, коммуникации с водителем, решения ИИ и действия пользователя. Такой подход позволяет представить историю ситуации в хронологическом виде.

Отдельное место занимает история коммуникаций. В интерфейсе должны отображаться диалоги через «MAX» и SMS, а также информация о голосовых коммуникациях и их транскриптах. Это позволяет логисту анализировать не только итоговое решение ИИ, но и контекст взаимодействия с водителем.

Важным требованием является наличие «explainability layer» (слой объяснимости ИИ). В интерфейсе должны быть доступны намерения, сущности, уверенность, задействованные правила и источник правды. Кроме того, пользователь должен иметь возможность понять, почему было принято конкретное решение и почему система отправила определенное

уведомление.

Система должна поддерживать обновление данных в реальном времени. Состояние рейса или инцидента может измениться после получения нового сообщения, изменения ETA или появления нового решения ИИ. Поэтому frontend взаимодействует с backend посредством WebSocket и получает актуальные события без необходимости постоянного полного обновления страницы.

Для выполнения операций пользователь должен иметь доступ к действиям управления инцидентом. В зависимости от состояния ситуации логист может просмотреть детали, изменить состояние обработки, эскалировать проблему или выполнить другое предусмотренное бизнес-логикой действие.

Помимо основной страницы обзора, интерфейс должен включать раздел мониторинга и административную панель. Мониторинг позволяет контролировать состояние и показатели работы системы. Администрирование предназначено для настройки порогов эскалации, правил повторных коммуникаций, параметров ИИ-агента и прочих системных процессов.

1.4 Постановка задачи индивидуального задания

В рамках frontend-направления требовалось разработать интерфейс логиста-наблюдателя. В состав задачи входили проектирование общей структуры приложения, реализация навигации, разработка страницы обзора инцидентов и рейсов, создание интерфейса детализации, отображение событий и коммуникаций, разработка «explainability-интерфейса» для решений ИИ, а также создание страниц мониторинга и администрирования.

В рамках задания также требовалось обеспечить взаимодействие пользовательского интерфейса с backend. Для получения и изменения данных используется API, а для синхронизации в реальном времени — WebSocket.

Вторая часть индивидуального задания была связана с разработкой сервиса взаимодействия через мессенджер «МАХ». Для него была определена архитектура отдельного сервиса, подключена библиотека «MAX Bot API» и сформирована структура обработчиков сообщений. На текущем этапе реализована базовая функциональность бота: обработка приветствий, команда «/help» и повтор отправленных пользователем сообщений. Полноценная интеграция с ИИ-диспетчером и основной бизнес-логикой системы является следующим этапом развития сервиса.

Таким образом, индивидуальное задание включало разработку пользовательского слоя системы для контроля работы ИИ-диспетчера и создание основы коммуникационного сервиса для взаимодействия с водителями через МАХ. Основные требования к интерфейсу, архитектуре и функциональности были сформированы с учетом общей концепции проекта и необходимости обеспечить взаимодействие frontend-компонента с backend и ИИ-компонентами системы.

2 Архитектура и технологические решения системы

2.1 Общая архитектура системы

Разрабатываемая система представляет собой набор взаимодействующих сервисов, каждый из которых отвечает за отдельную часть процесса интеллектуальной диспетчеризации. Такое разделение позволяет независимо развивать frontend, backend, ИИ-компоненты и сервисы обработки фоновых задач.

Основным источником данных для пользовательского интерфейса является backend-сервис. Он отвечает за обработку бизнес-логики, работу с данными о рейсах и инцидентах, управление состояниями объектов и взаимодействие с другими компонентами системы. Frontend получает необходимые данные через API и использует WebSocket для получения событий в реальном времени.

ИИ-компонент системы отвечает за обработку информации и принятие решений в рамках заданных сценариев. В качестве основной языковой модели используется GigaChat от Сбера. Для построения цепочек взаимодействия с ИИ применяется LangChain, а интеграция с GigaChat реализуется с помощью «langchain-gigachat». Langfuse используется для наблюдаемости и анализа ИИ-процессов.

Для хранения данных используется база данных PostgreSQL с расширением PostGIS. Применение PostGIS позволяет работать с географическими данными, связанными с местоположением транспортных средств и маршрутизацией. Redis используется для задач, требующих быстрого доступа к данным и организации взаимодействия между отдельными компонентами системы (как брокер сообщений).

Развертывание системы выполняется с использованием «Docker Compose». На текущем этапе система включает семь контейнеризованных микросервисов, взаимодействующих между собой в рамках общей инфраструктуры. Такой подход позволяет запускать все необходимые компоненты проекта в согласованном окружении и упрощает настройку

среды разработки.

Общая архитектура системы представлена на рисунке 3.

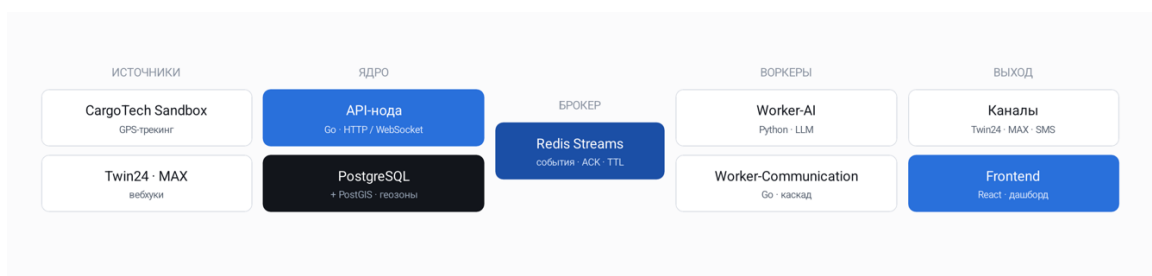


Рисунок 3 — Архитектура системы

Frontend-приложение взаимодействует с backend через два основных механизма. REST API используется для получения данных, выполнения операций с сущностями системы и изменения состояния объектов. WebSocket применяется для передачи событий в реальном времени. Например, после появления нового события, изменения состояния рейса или формирования нового инцидента backend может отправить соответствующее обновление в frontend.

Отдельным компонентом системы является сервис МАХ-бота. Он предназначен для взаимодействия с водителями через мессенджер «МАХ». Бот рассматривается как отдельный сервис, взаимодействующий с основной backend-системой. Такой подход позволяет отделить логику работы с внешним API мессенджера от основной бизнес-логики приложения.

Взаимодействие компонентов системы организовано таким образом, чтобы каждый сервис имел определенную зону ответственности. Backend отвечает за бизнес-логику и данные, ИИ-компоненты — за интеллектуальную обработку информации, frontend — за пользовательское взаимодействие, а МАХ-бот — за коммуникацию через мессенджер.

2.2 Технологический стек и организация разработки

При разработке системы использовался стек технологий, соответствующий требованиям проекта и задачам отдельных компонентов. Backend-часть системы разрабатывается на языке Go. Для хранения данных

используется PostgreSQL с расширением PostGIS, а Redis применяется в качестве высокопроизводительного хранилища и инфраструктурного компонента.

ИИ-часть системы построена с использованием GigaChat от Сбера. Для организации взаимодействия с языковой моделью используются LangChain и «langchain-gigachat». Langfuse применяется для трассировки и наблюдения за ИИ-процессами.

Frontend-приложение разрабатывается на React с использованием TypeScript. Для построения приложения используется TanStack Start. TanStack Query применяется для работы с серверными данными, выполнения запросов и управления состояниями загрузки и обновления. Для локального состояния интерфейса используется Zustand.

Для взаимодействия с backend используется WebSocket. Это позволяет получать изменения состояния системы без постоянного ручного обновления страницы. Визуальная часть приложения создается с использованием ShadcnUI и TailwindCSS. Такой подход позволяет использовать переиспользуемые компоненты интерфейса и поддерживать единый стиль приложения.

Сервис МАХ-бота разрабатывается отдельно от frontend-приложения. Для взаимодействия с мессенджером используется «MAX Bot API».

В проекте используется монорепозитарий, в котором размещаются основные компоненты системы: API, фоновые воркеры, frontend и сервис бота. Для разделения зон ответственности применяется метод «Code Owners» по направлениям Backend, ИИ и Frontend.

Разработка функциональности выполняется через feature-ветки. Изменения в основную ветку вносятся посредством Pull Request и проходят обязательное ревью. Прямые изменения основной ветки ограничены. Для единообразного оформления истории изменений используется единый формат коммитов («Conventional Commits»).

Для автоматизации проверки изменений используется CI. При создании Pull Request выполняются линтинг, тестирование и сборка контейнеров. Дополнительно применяется сквозной `trace_id`, позволяющий связывать события, проходящие через различные компоненты системы. Это упрощает диагностику проблем при обработке одного запроса или события несколькими сервисами.

Схема успешного CI-процесса и отрезок из истории репозитория представлены на рисунке 4.

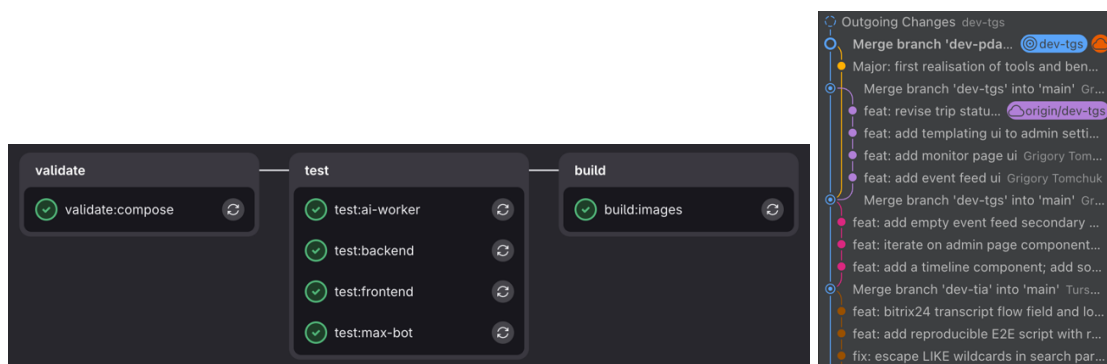


Рисунок 4 — Слева: схема конвейера сборки и тестирования в GitLab CI.

Справа: отрезок из истории системы контроля версий (ветка каждого разработчика происходит от и регулярно сливается с мастер-веткой)

Работа команды организована с использованием канбан-доски. Задачи распределяются между участниками команды и проходят через этапы планирования, разработки, проверки и завершения. Такой подход позволяет отслеживать текущий прогресс проекта и координировать работу frontend-, backend- и ИИ-направлений.

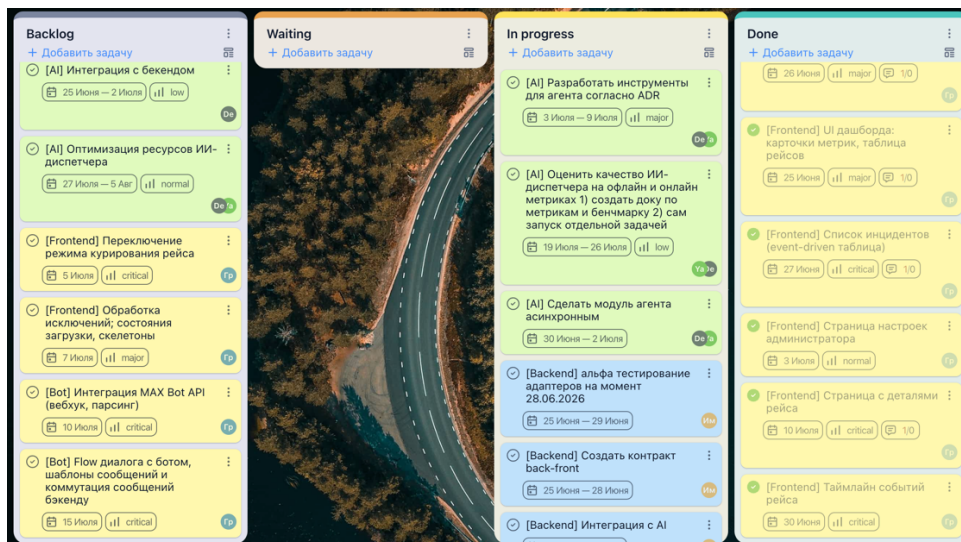


Рисунок 5 — Канбан-доска проекта

Планирование основных этапов работы представлено в виде диаграммы Ганта. Она позволяет отобразить последовательность разработки компонентов системы и взаимосвязь задач между различными направлениями. В частности, разработка frontend зависит от доступности API и WebSocket со стороны backend, а полноценная интеграция MAX-бота — от готовности механизмов взаимодействия с основной системой.

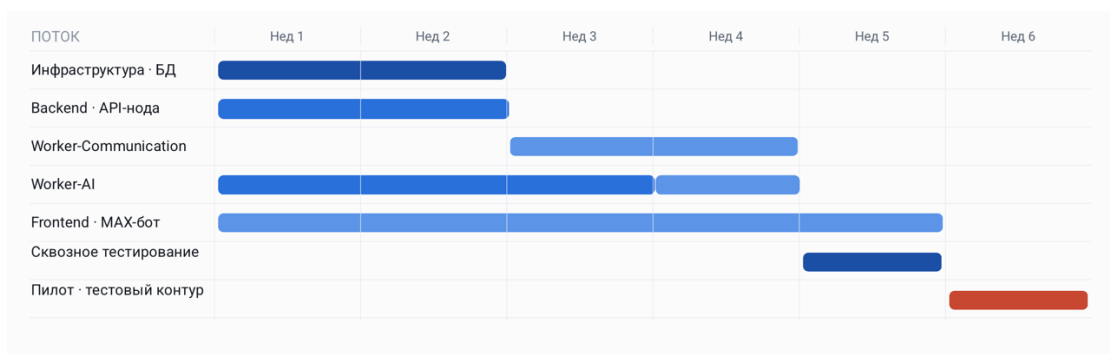


Рисунок 6 — Диаграмма Ганта разработки проекта

2.3 Архитектура frontend-компонента

Frontend-компонент системы предназначен для предоставления логисту единого интерфейса наблюдения за состоянием рейсов, инцидентов и ИИ-диспетчера.

Приложение построено на React и TypeScript. Использование TypeScript позволяет формализовать структуры данных, получаемых от API и WebSocket, и уменьшить вероятность ошибок при взаимодействии

между frontend и backend.

Vite и «TanStack Router» используются в качестве основы приложения. Они предоставляют необходимую инфраструктуру для построения современного React-приложения и организации маршрутизации. «TanStack Query» используется для работы с серверным состоянием: получения данных, кэширования, повторных запросов и синхронизации данных с backend.

Локальное состояние интерфейса, не являющееся серверными данными, управляется с использованием Zustand. К таким данным относятся состояние открытых панелей и модальных окон, выбранный инцидент, параметры интерфейса и другие пользовательские состояния.

Взаимодействие с backend разделяется на два основных типа. Для получения и изменения данных используются HTTP-запросы к REST API. Для получения событий в реальном времени используется WebSocket. После получения нового события frontend обновляет соответствующую часть интерфейса, не выполняя полную перезагрузку страницы.

Такой подход соответствует «event-driven» модели работы приложения. Например, получение нового сообщения от водителя может привести к появлению нового события в ленте событий, изменению состояния инцидента или обновлению показателей на странице обзора.

Интерфейс построен из переиспользуемых компонентов. Для реализации визуальных компонентов используется ShadcnUI, а TailwindCSS применяется для стилизации и адаптивного управления внешним видом элементов.

Архитектура frontend-компонента представлена на рисунке 7.

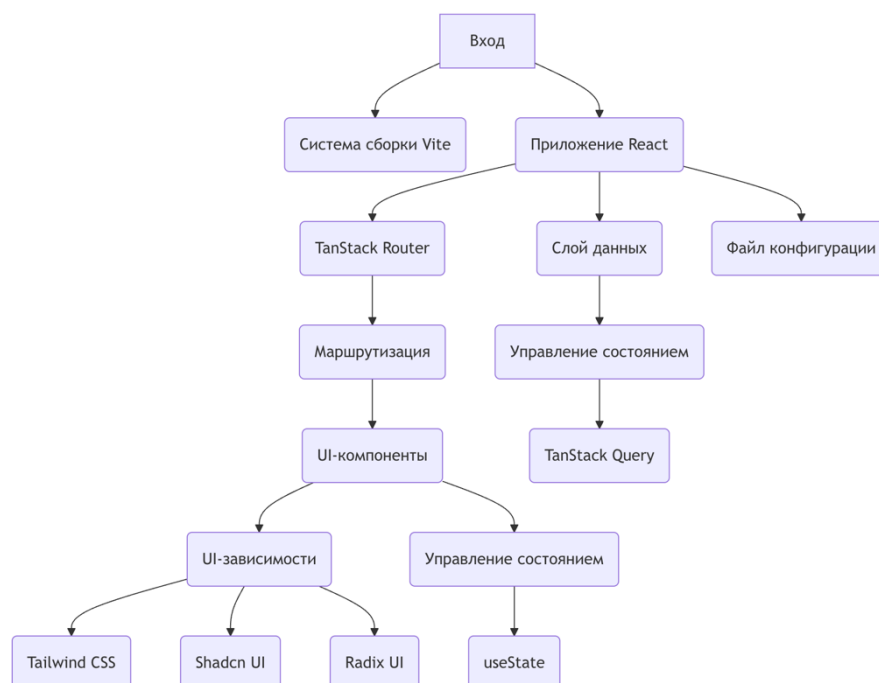


Рисунок 7 — Архитектура frontend-приложения

2.4 Архитектура сервиса МАХ-бота

МАХ-бот является отдельным сервисом, предназначенным для организации коммуникации между ИИ-диспетчером и водителем.

В качестве основного инструмента взаимодействия используется МАХ Bot API. Сервис получает события от мессенджера, передает их в соответствующие обработчики и формирует ответы пользователю. Архитектура предусматривает разделение обработчиков сообщений и дальнейшей бизнес-логики сервиса.

На текущем этапе работы над проектом была сформирована структура обработчиков сообщений и получен токен бота. Реализована базовая функциональность, включающая обработку приветственных сообщений, команды /help и повтор отправленных пользователем сообщений.

Полноценное взаимодействие МАХ-бота с ИИ-диспетчером на текущем этапе еще не реализовано. В дальнейшем бот должен стать одним из каналов коммуникации с водителем. Полученные от водителя сообщения будут передаваться в основную систему для обработки, а команды на отправку сообщений могут поступать от backend-сервиса.

Сервис МАХ-бота отделен от основного backend на уровне архитектуры. Это позволяет независимо развивать интеграцию с API мессенджера и не смешивать ее с основной бизнес-логикой диспетчеризации.

На рисунке 8 представлена схема взаимодействия водителя, МАХ-бота, backend и ИИ-диспетчера.

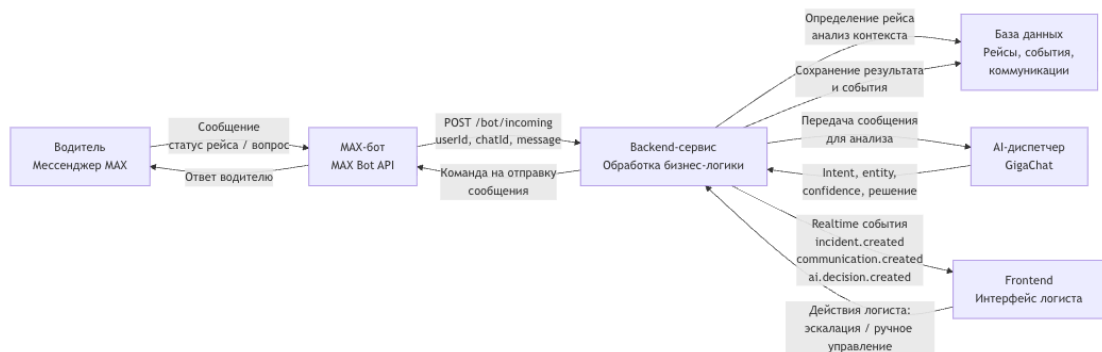


Рисунок 8 — Архитектура взаимодействия МАХ-бота с основной системой

Таким образом, выбранная архитектура обеспечивает разделение ответственности между компонентами системы. Frontend отвечает за отображение и взаимодействие с логистом, backend — за бизнес-логику и данные, ИИ-компоненты — за интеллектуальную обработку информации, а МАХ-бот — за взаимодействие с водителем через внешний коммуникационный канал.

3 Разработка пользовательского веб-интерфейса и сервиса МАХ-бота

3.1 Разработка структуры и основных компонентов пользовательского интерфейса

Одной из основных задач в рамках индивидуального задания являлась разработка пользовательского веб-интерфейса для логиста-наблюдателя. При проектировании интерфейса учитывалось, что система предназначена не для ручного управления каждым рейсом, а для контроля работы ИИ-диспетчера и обработки ситуаций, требующих внимания специалиста.

Основным объектом взаимодействия в интерфейсе являются инциденты. В отличие от классических систем управления перевозками, где пользователь работает с большим перечнем рейсов, в разработанном интерфейсе основной акцент сделан на отображении отклонений, рисков и решений искусственного интеллекта.

Общая структура приложения включает несколько основных разделов:

1. обзор текущего состояния системы;
2. мониторинг показателей работы;
3. административные настройки;
4. детализацию отдельных инцидентов и рейсов.

Для обеспечения удобной навигации была реализована общая оболочка приложения. Она включает боковую панель навигации, область основного содержимого и дополнительный блок отображения последних событий системы. В нижней части боковой панели отображается состояние подключения к системе и статус синхронизации данных. Код, отвечающий за отображение главной оболочки интерфейса представлен в приложении Б.

Использование событийной модели работы приложения позволяет отображать изменения состояния системы в реальном времени. Например, после появления нового инцидента или изменения состояния рейса

соответствующая информация обновляется без необходимости полного обновления страницы.

На рисунке 9 представлен общий вид интерфейса приложения с основной структурой навигации.

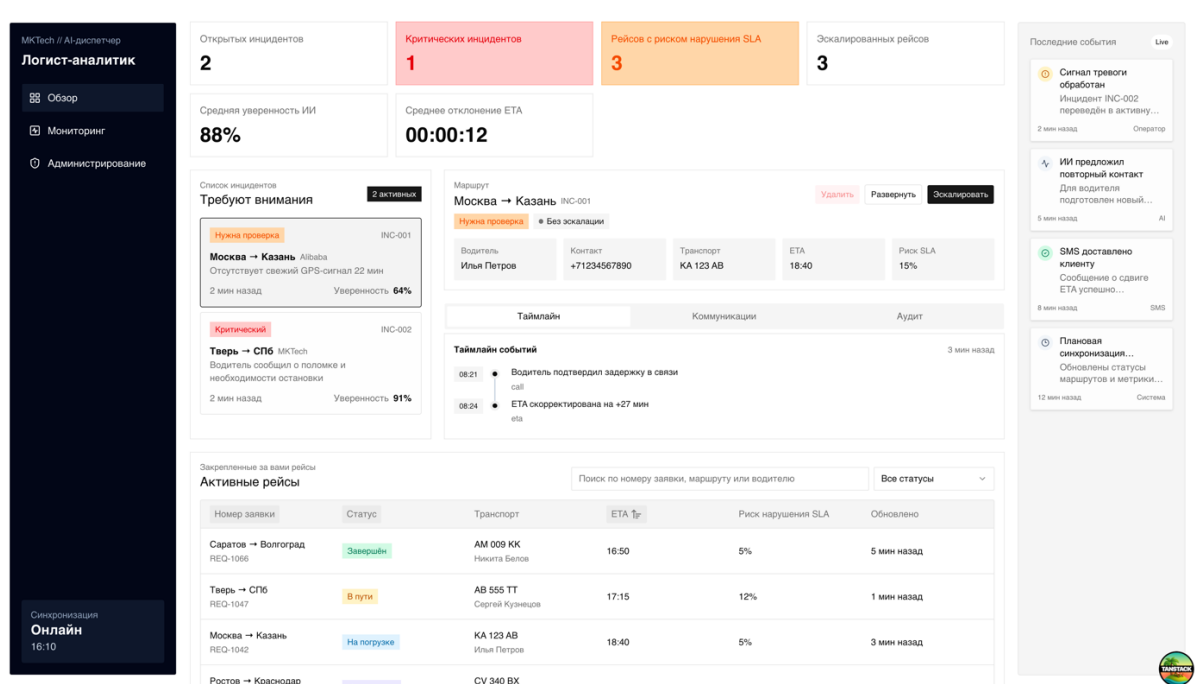


Рисунок 9 — Общая структура пользовательского интерфейса системы

Для реализации frontend-части использовались React и TypeScript. Компонентный подход React позволил разделить интерфейс на независимые элементы и обеспечить повторное использование компонентов. TypeScript использовался для описания структур данных и повышения надежности взаимодействия с backend API.

Работа с серверными данными реализована через TanStack Query. Данный инструмент используется для выполнения запросов, управления состоянием загрузки данных, кэширования и синхронизации информации с backend. Для управления локальным состоянием интерфейса применяется Zustand.

Для обмена данными в режиме реального времени используется WebSocket. Получаемые события обрабатываются frontend-приложением и используются для обновления соответствующих элементов интерфейса: списка инцидентов, ленты событий и показателей системы.

Визуальная часть приложения реализована с использованием ShadcnUI и TailwindCSS. Такой подход позволил создать набор переиспользуемых UI-компонентов и обеспечить единый стиль интерфейса. Примеры кода компонентов интерфейса приведены в приложении Б.

3.2 Реализация страницы обзора и интерфейса работы с инцидентами

Основным экраном системы является страница «Обзор». Ее задача заключается в предоставлении логисту оперативной информации о текущем состоянии диспетчеризации и выделении ситуаций, требующих внимания.

На странице отображается сводная информация по работе системы:

- количество открытых инцидентов;
- количество критических инцидентов;
- количество рейсов с риском нарушения SLA;
- количество эскалированных рейсов;
- средняя уверенность ИИ;
- среднее отклонение ETA.

Данные показатели позволяют пользователю быстро оценить текущее состояние перевозок и определить наличие проблемных областей.

Основная область страницы содержит список активных инцидентов. Для каждого инцидента отображается информация о его состоянии, критичности и связанных с ним показателях. При выборе конкретного элемента открывается дополнительная область детализации.

В панели детализации выбранного инцидента отображаются основные сведения о проблемной ситуации, а также доступны действия управления (см. в приложении А). В зависимости от состояния инцидента пользователь может выполнить операции изменения, удаления или эскалации.

Для более глубокого анализа ситуации предусмотрены три основные вкладки:

- лента событий рейса и инцидента;
- история коммуникаций с водителем;
- аудит решений искусственного интеллекта.

Лента событий представляет собой временную последовательность действий и изменений состояния системы. В ней могут отображаться изменение статуса рейса, изменение ЕТА, получение новых сообщений, результаты ИИ-анализа и действия пользователя.

История коммуникаций предназначена для отображения взаимодействия с водителем. В ней содержатся сообщения из мессенджера «MAX», SMS, а также данные о голосовых коммуникациях и транскриптах разговоров.

Аудит решений ИИ предназначен для отображения информации, связанной с процессом принятия решения искусственным интеллектом. Пользователь может увидеть распознанное намерение, извлеченные сущности, значение уверенности и причины выполнения определенного действия (см. в приложении А).

На рисунке 10 представлен интерфейс страницы обзора с отображением ключевых показателей и списка активных инцидентов.

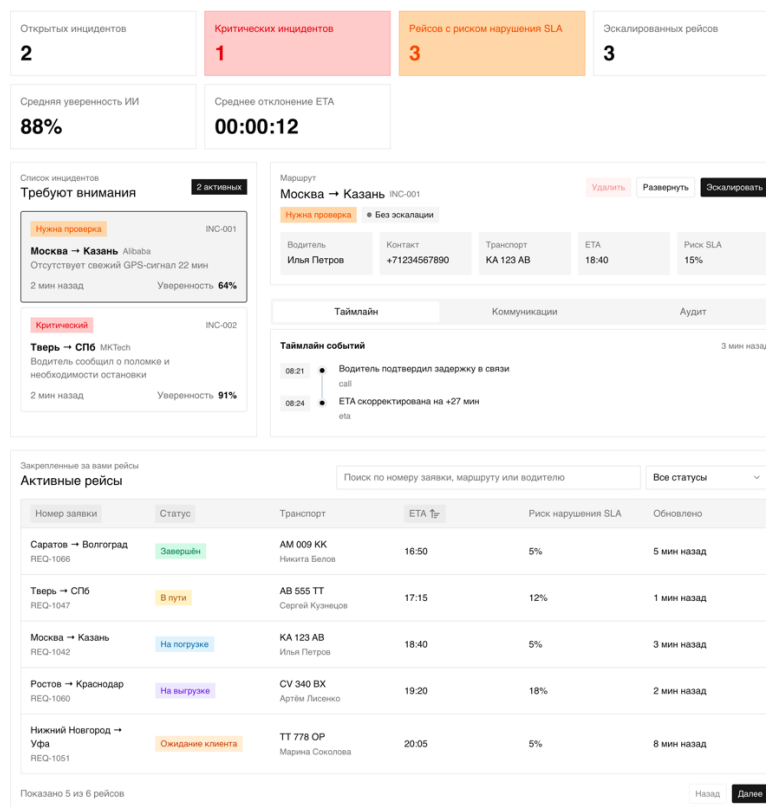


Рисунок 10 — Страница «Обзор»

Дополнительно в интерфейсе предусмотрены модальные окна для просмотра расширенной информации. Одно из них предназначено для детализации инцидента, другое — для просмотра полной информации о связанном рейсе (см. в приложении А).

Ниже основной области отображается таблица активных рейсов, закрепленных за логистом. В таблице представлены:

- номер заявки;
- направление движения;
- текущий статус;
- номер транспортного средства;
- ETA;
- риск нарушения SLA.

Таким образом, страница обзора объединяет основные сценарии работы логиста: оценку состояния системы, поиск проблемных ситуаций и переход к детальному анализу конкретного рейса.

3.3 Реализация страниц мониторинга и администрирования

Помимо основного рабочего экрана были разработаны интерфейсы мониторинга и административного управления системой.

Страница мониторинга предназначена для контроля технического состояния и эффективности работы ИИ-диспетчера. В отличие от страницы обзора, которая ориентирована на обработку инцидентов, данный раздел предоставляет информацию о работе самой системы.

На странице мониторинга отображаются:

- длительность обработки запросов;
- количество новых событий;
- показатели производительности;
- состояние правил автоматического алертирования (тревоги).

Для контроля состояния системы используются правила, которые позволяют определить, когда определенные показатели выходят за допустимые значения. Например, может быть настроено предупреждение при увеличении времени обработки событий или снижении стабильности отдельных компонентов.

На рисунке 11 представлен интерфейс страницы мониторинга.

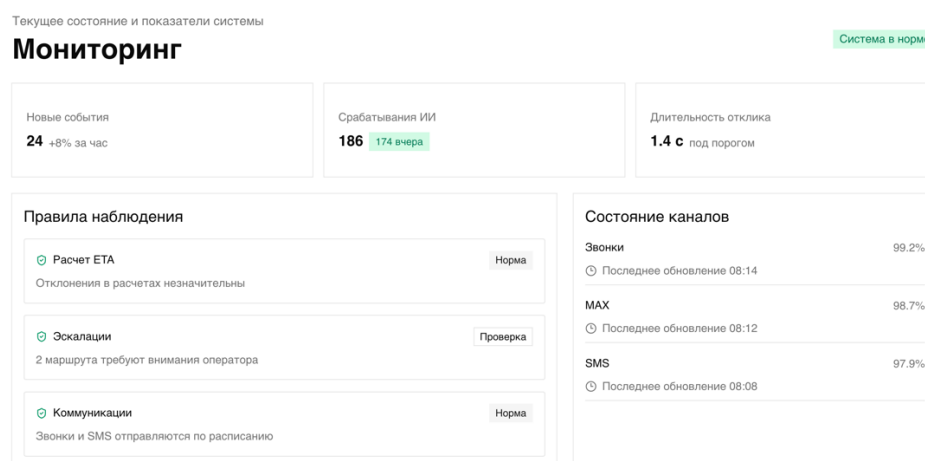


Рисунок 11 — Страница «Мониторинг»

Административная панель предназначена для настройки параметров работы системы. Она включает несколько разделов:

- эскалации;

- повторы;
- ИИ-агент;
- процессы.

Раздел «Эскалации» позволяет настраивать пороговые значения, при достижении которых система должна формировать предупреждение или передавать ситуацию логисту. Например, могут задаваться параметры риска нарушения SLA или минимальный уровень уверенности ИИ.

Раздел «Повторы» предназначен для настройки правил повторного взаимодействия с водителем. В нем задаются интервалы между повторными звонками, сообщениями в мессенджере и SMS. Дополнительно предусмотрена возможность настройки временных периодов активности правил (см. в приложении А).

Раздел «ИИ-агент» содержит параметры, связанные с настройкой работы искусственного интеллекта (см. в приложении А).

Раздел «Процессы» включает дополнительные системные настройки: параметры логирования, версионирование правил и другие технические параметры (см. в приложении А).

Для всех разделов административной панели предусмотрены операции сохранения и сброса настроек.

На рисунке 12 представлен интерфейс административной панели системы.

Настройки правил, коммуникаций и ИИ-агента

Администрирование

Черновик локальных правил

[Сбросить](#) [Сохранить](#)

Активных правил 3	Каналов повтора 3	Порог автодействий 88%	Автозаккрытие 24 ч
-----------------------------	-----------------------------	----------------------------------	------------------------------

Эскалации	Повторы	ИИ-агент	Процессы																				
Пороги создания и усиления инцидентов Пороги эскалаций <table border="1"> <thead> <tr> <th>Правило</th> <th>Предупреждение</th> <th>Алерт</th> <th>Действие</th> <th>Активно</th> </tr> </thead> <tbody> <tr> <td>Риск нарушения SLA Прогноз отклонения ETA</td> <td>12 %</td> <td>25 %</td> <td>Создать инцидент и уведомить ло</td> <td>☒</td> </tr> <tr> <td>Потеря GPS-сигнала Нет свежей телеметрии</td> <td>15 мин</td> <td>30 мин</td> <td>Позвонить водителю, затем эскал</td> <td>☒</td> </tr> <tr> <td>Риск недоступности водителя Неуспешные контакты подряд</td> <td>2 поп.</td> <td>4 поп.</td> <td>Подключить старшего диспетчера</td> <td>☒</td> </tr> </tbody> </table>				Правило	Предупреждение	Алерт	Действие	Активно	Риск нарушения SLA Прогноз отклонения ETA	12 %	25 %	Создать инцидент и уведомить ло	☒	Потеря GPS-сигнала Нет свежей телеметрии	15 мин	30 мин	Позвонить водителю, затем эскал	☒	Риск недоступности водителя Неуспешные контакты подряд	2 поп.	4 поп.	Подключить старшего диспетчера	☒
Правило	Предупреждение	Алерт	Действие	Активно																			
Риск нарушения SLA Прогноз отклонения ETA	12 %	25 %	Создать инцидент и уведомить ло	☒																			
Потеря GPS-сигнала Нет свежей телеметрии	15 мин	30 мин	Позвонить водителю, затем эскал	☒																			
Риск недоступности водителя Неуспешные контакты подряд	2 поп.	4 поп.	Подключить старшего диспетчера	☒																			

3.4 Разработка сервиса взаимодействия через мессенджер МАХ

В рамках индивидуального задания также выполнялась работа над сервисом взаимодействия через мессенджер МАХ. Данный компонент предназначен для организации коммуникации между ИИ-диспетчером и водителем.

Для реализации используется официальная библиотека «МАХ Bot API». На начальном этапе был получен токен бота, настроено подключение к API и разработана базовая структура обработчиков сообщений.

В текущей версии реализованы основные тестовые сценарии:

- обработка приветственных сообщений;
- выполнение команды «/help»;
- получение сообщения пользователя и отправка ответного сообщения.

Данные сценарии позволяют проверить корректность взаимодействия с API мессенджера и подготовить основу для дальнейшей интеграции.

Полноценная логика взаимодействия с ИИ-диспетчером на текущем этапе находится в процессе разработки. В дальнейшем бот должен использоваться как один из каналов связи с водителем: получать сообщения, передавать их в основную систему для анализа и отправлять пользователю сформированные ИИ-ответы. Наиболее значимые фрагменты кода МАХ-бота приведены в приложении Б.

Для взаимодействия между ботом и backend предусмотрена отдельная интеграция через API. Backend будет отвечать за принятие решений и формирование команд, а бот — за доставку сообщений через внешний канал коммуникации.

На рисунке 13 представлен интерфейс взаимодействия с ботом в мессенджере «МАХ».

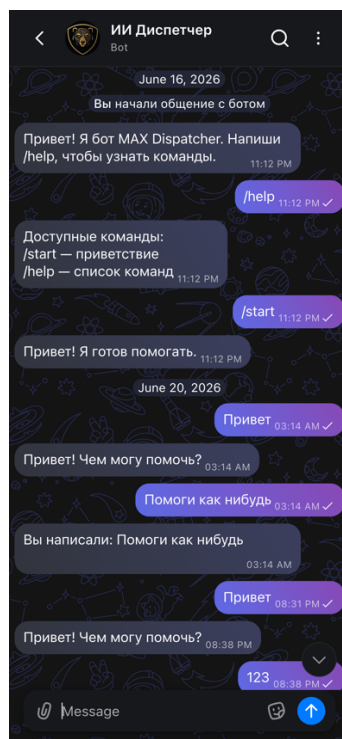


Рисунок 13 — Интерфейс взаимодействия с ботом в мессенджере «МАХ»

Таким образом, в рамках разработки были сформированы основные пользовательские интерфейсы системы и создана основа коммуникационного сервиса через МАХ. Реализованные компоненты обеспечивают необходимый пользовательский слой для контроля работы ИИ-диспетчера и дальнейшего расширения функциональности системы.

4 Результаты разработки и перспективы дальнейшего развития системы

4.1 Реализованные результаты разработки

В ходе выполнения индивидуального задания были разработаны основные компоненты пользовательского слоя системы интеллектуальной диспетчеризации логистических рейсов.

В рамках frontend-направления был создан пользовательский веб-интерфейс для логиста-наблюдателя. Основной задачей интерфейса является предоставление актуальной информации о состоянии системы, отображение возникающих инцидентов и обеспечение возможности анализа решений ИИ-диспетчера.

В текущей версии приложения реализована общая структура интерфейса с навигационной панелью, областью отображения основного содержимого, блоком последних событий и индикаторами состояния подключения к системе. Данная структура является основой для дальнейшего расширения функциональности приложения.

Реализована страница «Обзор», которая является основным рабочим экраном логиста. На ней отображаются ключевые показатели состояния системы. Также реализовано отображение списка активных инцидентов и области детализации выбранной ситуации. Пользователь может изучить информацию о рейсе, просмотреть историю событий, коммуникации с водителем и данные о процессе принятия решений искусственным интеллектом. Особое внимание при разработке было уделено реализации слоя «explainability». В интерфейсе предусмотрена возможность отображения информации, необходимой для анализа решений ИИ: распознанного намерения, извлеченных сущностей, уровня уверенности и причин выполнения определенного действия. Это позволяет сделать работу ИИ-диспетчера более прозрачной для пользователя.

Дополнительно были разработаны страницы мониторинга и администрирования. Страница мониторинга предназначена для анализа технических и эксплуатационных показателей системы. На ней

отображаются параметры работы сервисов, количество событий и состояние правил контроля.

Административная панель предоставляет интерфейсы для настройки параметров системы. В ней предусмотрены разделы управления правилами эскалации, настройками повторных коммуникаций, параметрами ИИ-агента и системными процессами.

В рамках разработки МАХ-бота была создана основа отдельного сервиса взаимодействия с мессенджером. Было выполнено подключение к API мессенджера, получен токен бота и реализована базовая структура обработчиков сообщений.

4.2 Взаимодействие разработанных компонентов с общей системой

Разработанные frontend-компоненты являются частью общей архитектуры системы и взаимодействуют с backend-сервисами через определенные интерфейсы обмена данными.

Получение информации о рейсах, инцидентах, событиях и настройках системы осуществляется через REST API. Такой подход используется для загрузки основных данных, получения информации о сущностях системы и выполнения пользовательских действий.

Для непрерывного обеспечения актуального состояния интерфейса используется WebSocket-соединение. Благодаря этому frontend может получать события от backend в режиме реального времени. Например, после появления нового инцидента, изменения состояния рейса или получения нового сообщения от водителя соответствующая информация может быть автоматически отображена в интерфейсе.

Использование событийного подхода особенно важно для системы диспетчеризации, так как состояние рейсов может изменяться в процессе выполнения перевозки. Постоянное обновление данных через периодические запросы увеличивало бы нагрузку на систему и не обеспечивало бы необходимую скорость реакции.

Архитектура взаимодействия frontend и backend построена таким

образом, чтобы разделить ответственность между компонентами. Frontend отвечает за отображение информации и пользовательские действия, а backend выполняет обработку данных и реализацию бизнес-логики.

МАХ-бот является отдельным сервисом и также взаимодействует с основной системой через REST API backend-сервиса. Такое разделение позволяет использовать мессенджер как независимый канал коммуникации с водителем. В дальнейшем сообщения от водителя будут передаваться в ИИ-компонент для анализа, а результаты обработки будут возвращаться через бота.

На текущем этапе API и WebSocket-интерфейсы находятся в активной разработке backend-командой. Основные механизмы взаимодействия определены, а дальнейшее развитие системы связано с расширением набора доступных данных и подключением дополнительных сценариев обработки.

На рисунке 14 представлена схема взаимодействия разработанных компонентов с остальными сервисами системы.

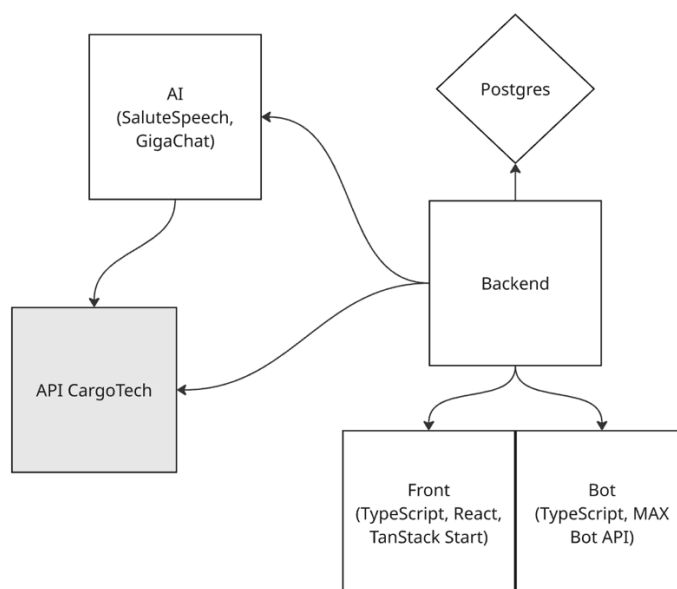


Рисунок 14 — Взаимодействие frontend, МАХ-бота, backend и ИИ-компонентов

4.3 Текущий статус проекта и дальнейшее развитие

На текущем этапе проекта реализована основа пользовательского

интерфейса системы и сформирована архитектура основных компонентов. Разработаны ключевые экраны приложения, определены принципы взаимодействия с backend и подготовлена структура сервиса МАХ-бота.

Основные дальнейшие направления развития системы связаны с расширением интеграции между компонентами и переходом от тестовых сценариев к полноценной эксплуатации.

Для frontend-направления дальнейшая разработка включает:

- подключение полного набора backend API;
- интеграцию всех WebSocket-событий;
- отображение реальных данных о рейсах и инцидентах;
- расширение интерфейсов анализа решений ИИ;
- улучшение пользовательского опыта работы логиста.

Для МАХ-бота дальнейшими этапами являются:

- интеграция с backend-сервисом;
- обработка реальных сценариев общения с водителями;
- передача сообщений в ИИ-компонент;
- получение и отправка автоматических ответов;
- поддержка дополнительных типов сообщений.

После завершения интеграции всех компонентов система должна пройти этап подготовки к пилотной эксплуатации. В рамках пилота планируется проверить эффективность работы ИИ-диспетчера на реальных перевозочных процессах и оценить достижение целевых показателей проекта.

Ключевыми метриками эффективности системы являются:

- доля рейсов без ручного вмешательства логиста;
- успешность взаимодействия ИИ-агента с водителями;
- точность прогнозирования ЕТА;
- стоимость обработки одного рейса;
- снижение временных затрат логистов.

Разработанная архитектура позволяет постепенно расширять функциональность системы без необходимости изменения основных компонентов. Разделение frontend, backend, ИИ-модулей и коммуникационных сервисов обеспечивает возможность независимого развития отдельных частей продукта.

Таким образом, в ходе выполнения производственной практики были реализованы основные элементы пользовательского слоя системы интеллектуальной диспетчеризации и создана техническая основа для дальнейшего развития «ИИ-диспетчера рейсов MKTech».

ЗАКЛЮЧЕНИЕ

В рамках производственной практики была выполнена разработка пользовательского веб-интерфейса и сервиса взаимодействия через мессенджер «МАХ» для системы интеллектуальной диспетчеризации логистических рейсов «ИИ-диспетчер рейсов МКTech».

В ходе работы была изучена предметная область автоматизации логистической диспетчеризации и определены основные требования к системе. Было установлено, что ключевой задачей разрабатываемого решения является снижение нагрузки на логистов за счет автоматизации контроля рейсов, анализа отклонений и обработки типовых ситуаций с использованием искусственного интеллекта.

В рамках frontend-направления был разработан интерфейс логиста-наблюдателя, ориентированный на контроль работы ИИ-диспетчера и обработку исключительных ситуаций. Были реализованы основные элементы приложения: общая структура интерфейса, страница обзора активных инцидентов и рейсов, интерфейс детализации ситуаций, страницы мониторинга и административных настроек.

Особое внимание было уделено реализации механизмов объяснимости решений искусственного интеллекта. В интерфейсе предусмотрено отображение информации о причинах принятия решений ИИ, включая распознанные намерения, уровень уверенности, активированные правила и историю событий. Это позволяет пользователю не только получать результат работы системы, но и анализировать логику его формирования.

Также была разработана архитектура взаимодействия frontend-приложения с backend-сервисами. Для получения и изменения данных предусмотрено использование REST API, а для обновления информации в режиме реального времени — WebSocket-соединение. Такой подход соответствует событийной модели работы системы и позволяет отображать актуальное состояние рейсов и инцидентов без необходимости постоянного обновления интерфейса.

В рамках разработки сервиса МАХ-бота была сформирована архитектура отдельного коммуникационного компонента, выполнено подключение к API мессенджера «МАХ» и реализована базовая функциональность обработки сообщений. Созданная структура является основой для дальнейшей интеграции бота с ИИ-диспетчером и использования его в качестве канала взаимодействия с водителями.

В процессе разработки были использованы современные технологии создания веб-приложений и распределенных систем: React, TypeScript, TanStack Router, TanStack Query, Zustand, WebSocket, ShadcnUI, Tailwind CSS, а также инструменты взаимодействия с ИИ-компонентами GigaChat, LangChain и Langfuse. Применение выбранного технологического стека позволило создать гибкую архитектуру, пригодную для дальнейшего расширения функциональности системы.

На текущем этапе проекта реализованы основные пользовательские интерфейсы и подготовлена архитектурная основа для дальнейшего развития продукта. Следующими этапами являются подключение полного набора API и WebSocket-событий, расширение функциональности МАХ-бота, интеграция всех компонентов системы и подготовка решения к пилотной эксплуатации.

Таким образом, в ходе выполнения производственной практики были выполнены поставленные задачи по разработке пользовательского слоя системы интеллектуальной диспетчеризации и созданию основы коммуникационного сервиса для взаимодействия с участниками перевозочного процесса. Полученные результаты обеспечивают дальнейшее развитие системы и подготовку к ее использованию в реальных логистических процессах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Таненбаум, Э. Современные операционные системы / Э. Таненбаум, Х. Бос. — 4-е изд. — СПб.: Питер, 2021. — 1120 с.
2. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. — 10-е изд. — М.: Вильямс, 2019. — 816 с.
3. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. — М.: Вильямс, 2022. — 544 с.
4. React Documentation: официальная документация библиотеки React / Meta Platforms, Inc. — URL: <https://react.dev/> (дата обращения: 19.07.2026).
5. TypeScript Documentation: руководство по языку TypeScript / Microsoft. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 19.07.2026).
6. MAX Bot API: документация по разработке чат-ботов для мессенджера MAX / VK Tech. — URL: <https://dev.max.ru/docs/chatbots/bots-coding/js> (дата обращения: 13.07.2026).
7. GigaChat API: документация по работе с генеративными моделями GigaChat / Сбер. — URL: <https://developers.sber.ru/docs/ru/gigachat/> (дата обращения: 20.07.2026).

ПРИЛОЖЕНИЕ А

Реализованные экраны интерфейса

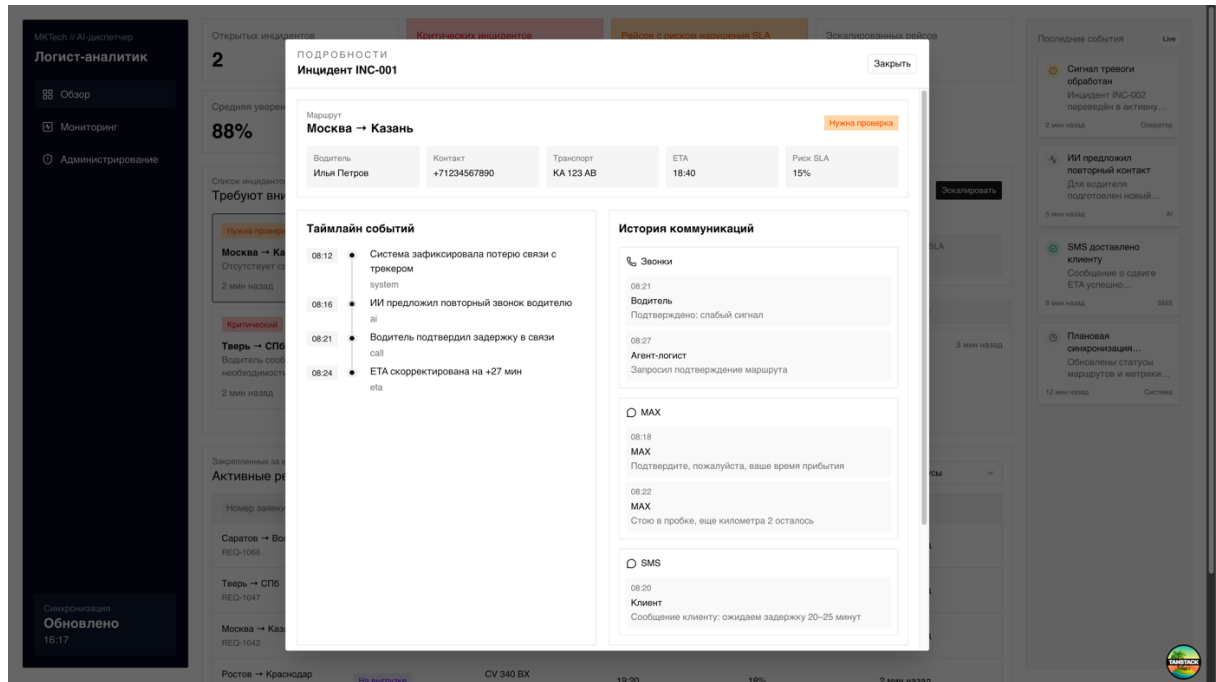


Рисунок 15 — Модальное окно детализации инцидента

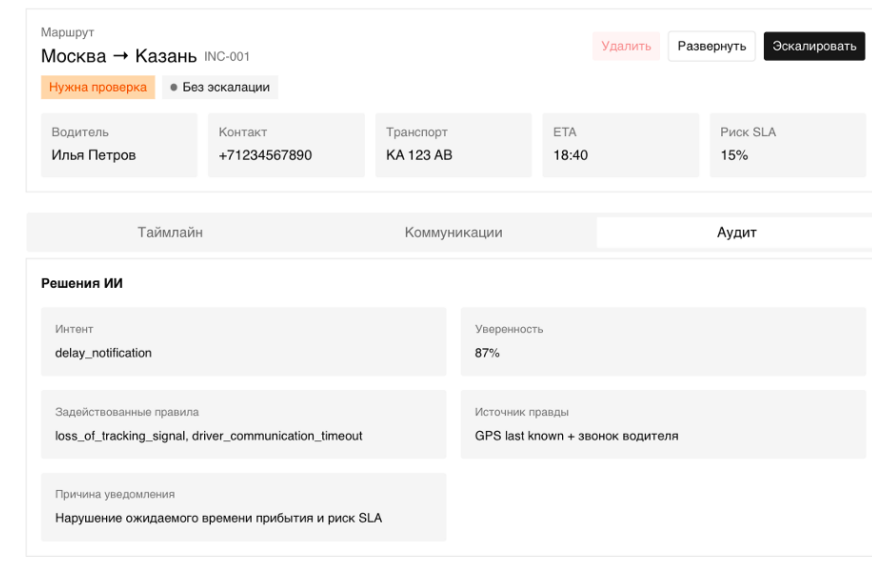


Рисунок 16 — Вкладка с показателями ИИ-агента, полезными для аудита принятых решений

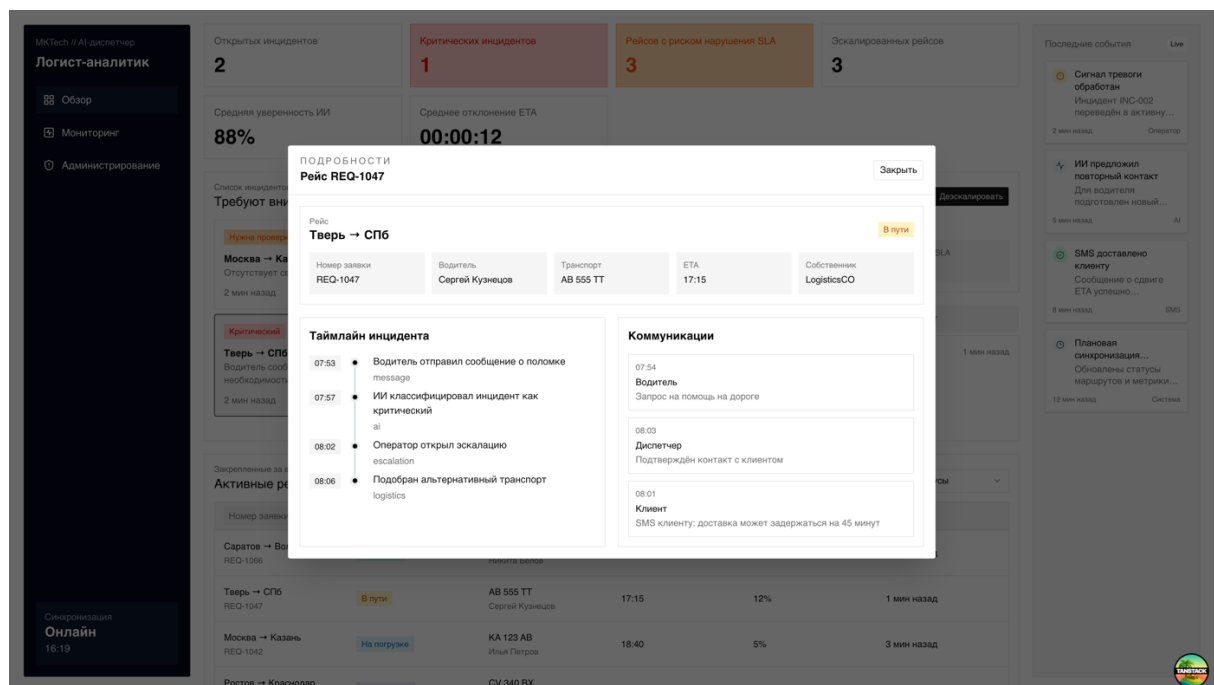


Рисунок 17 — Модальное окно детализации рейса

Настройки правил, коммуникаций и ИИ-агента

Администрирование

Черновик локальных правил

[Сбросить](#) [Сохранить](#)

Активных правил

3

Каналов повтора

3

Порог автодействий

88%

Автозаккрытие

24 ч

Эскалации	Повторы	ИИ-агент	Процессы
Звонок Ожидание первого ответа 3 мин Повтор через 5 мин Максимум попыток 3 поп. Тихие часы 22:00-07:00 Резервный ответственный Дежурный логист Шаблоны коммуникации Несколько шаблонов по времени суток и типу контакта Добавить шаблон Название шаблона Удалить Дневной контроль Время суток 08:00-20:00 Описание Контроль по ходу маршрута в рабо Название шаблона Удалить Ночной контакт Время суток 22:00-06:00 Описание Только по критическим событиям	Сообщение в MAX Ожидание первого ответа 2 мин Повтор через 7 мин Максимум попыток 2 поп. Тихие часы Без ограничений Резервный ответственный Оператор смены Шаблоны коммуникации Несколько шаблонов по времени суток и типу контакта Добавить шаблон Название шаблона Удалить Оперативный статус Время суток 08:00-22:00 Описание Короткие статусы по маршруту и Е Название шаблона Удалить Срочный шаблон Время суток Любое время Описание Пороговая коммуникация при заде	SMS Ожидание первого ответа 8 мин Повтор через 15 мин Максимум попыток 2 поп. Тихие часы 23:00-06:00 Резервный ответственный Старший диспетчер Шаблоны коммуникации Несколько шаблонов по времени суток и типу контакта Добавить шаблон Название шаблона Удалить Уведомление о задержке Время суток 08:00-21:00 Описание Сообщение клиенту о сдвиге ETA Название шаблона Удалить Ночная предосторожность Время суток 22:00-06:00 Описание Минимум сообщений, только важн	

Рисунок 18 — Страница «Администрирование», вкладка «Повторы»

Настройки правил, коммуникаций и ИИ-агента

Администрирование

Черновик локальных правил

Активных правил
3

Каналов повтора
3

Порог автодействий
88%

Автозаккрытие
24 ч

Эскалации

Повторы

ИИ-агент

Процессы

Пороги уверенности и границы автономности

Параметры ИИ-агента

Автодействия ИИ
Минимальная уверенность для звонка или сообщения без ревью.

88

88%

Ручная проверка
Ниже этого порога решение уходит оператору.

72

72%

Схожесть инцидентов
Порог поиска похожих кейсов для подсказок диспетчеру.

80

80%

Окно контекста
Сколько часов истории рейса учитывать агенту.

18

18ч

Рисунок 19 — Страница «Администрирование», вкладка «ИИ-агент»

Настройки правил, коммуникаций и ИИ-агента

Администрирование

Черновик локальных правил

Активных правил
3

Каналов повтора
3

Порог автодействий
88%

Автозаккрытие
24 ч

Эскалации

Повторы

ИИ-агент

Процессы

Общие процессы
Команда по умолчанию
Дневная смена
Автозаккрытие после стабилизации
24 ч
Режим ручного ревью
Только низкая уверенность

Контроль качества

☒ Версионирование правил

Включено

☒ Лог объяснений ИИ

90 дней

☒ Автотест сценариев

Перед публикацией

Рисунок 20 — Страница «Администрирование», вкладка «Процессы»

ПРИЛОЖЕНИЕ Б

Фрагменты кода реализованных сервисов

Листинг 1 — main.tsx (точка входа в приложение)

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query"
import { RouterProvider, createRouter } from "@tanstack/react-router"

import ReactDOM from "react-dom/client"
import { TooltipProvider } from "@components/ui/tooltip"
import { routeTree } from "./routeTree.gen"

const queryClient = new QueryClient()

const router = createRouter({
  routeTree,
  defaultPreload: "intent",
  scrollRestoration: true,
})

declare module "@tanstack/react-router" {
  interface Register {
    router: typeof router
  }
}

const rootElement = document.getElementById("app")!

if (!rootElement.innerHTML) {
  const root = ReactDOM.createRoot(rootElement)
  root.render(
    <QueryClientProvider client={queryClient}>
      <TooltipProvider delayDuration={1000}>
        <RouterProvider router={router} />
      </TooltipProvider>
    </QueryClientProvider>,
  )
}
```

Листинг 2 — globals.css (CSS-файл конфигурации TailwindCSS и ShadcnUI)

```
@import "tailwindcss";
@import "tw-animate-css";
@import "shadcn/tailwind.css";
/* @import "@fontsource-variable/jetbrains-mono"; */

@custom-variant dark (&:is(.dark *));

@theme {
  --text-xs: 0.9em;
  --text-sm: 1em;
  --text-base: 1.0625em;
  --text-2xl: 1.33em;
  --text-3xl: 2.25em;
  --spacing: 0.23rem;
}

:root {
  --background: oklch(1 0 0);
  --foreground: oklch(0.145 0 0);
}
```

```

--card: oklch(1 0 0);
--card-foreground: oklch(0.145 0 0);
--popover: oklch(1 0 0);
--popover-foreground: oklch(0.145 0 0);
--primary: oklch(0.205 0 0);
--primary-foreground: oklch(0.985 0 0);
--secondary: oklch(0.97 0 0);
--secondary-foreground: oklch(0.205 0 0);
--muted: oklch(0.97 0 0);
--muted-foreground: oklch(0.556 0 0);
--accent: oklch(0.94 0 0);
--accent-foreground: oklch(0.205 0 0);
--destructive: oklch(0.577 0.245 27.325);
--border: oklch(0.922 0 0);
--input: oklch(0.922 0 0);
--ring: oklch(0.708 0 0);
--chart-1: oklch(0.87 0 0);
--chart-2: oklch(0.556 0 0);
--chart-3: oklch(0.439 0 0);
--chart-4: oklch(0.371 0 0);
--chart-5: oklch(0.269 0 0);
--radius: 0.125rem;
--sidebar: oklch(0.985 0 0);
--sidebar-foreground: oklch(0.145 0 0);
--sidebar-primary: oklch(0.205 0 0);
--sidebar-primary-foreground: oklch(0.985 0 0);
--sidebar-accent: oklch(0.97 0 0);
--sidebar-accent-foreground: oklch(0.205 0 0);
--sidebar-border: oklch(0.922 0 0);
--sidebar-ring: oklch(0.708 0 0);
}

@theme inline {
  --font-heading: var(--font-mono);
  --font-mono: sans-serif;
  --color-sidebar-ring: var(--sidebar-ring);
  --color-sidebar-border: var(--sidebar-border);
  --color-sidebar-accent-foreground: var(--sidebar-accent-foreground);
  --color-sidebar-accent: var(--sidebar-accent);
  --color-sidebar-primary-foreground: var(--sidebar-primary-foreground);
  --color-sidebar-primary: var(--sidebar-primary);
  --color-sidebar-foreground: var(--sidebar-foreground);
  --color-sidebar: var(--sidebar);
  --color-chart-5: var(--chart-5);
  --color-chart-4: var(--chart-4);
  --color-chart-3: var(--chart-3);
  --color-chart-2: var(--chart-2);
  --color-chart-1: var(--chart-1);
  --color-ring: var(--ring);
  --color-input: var(--input);
  --color-border: var(--border);
  --color-destructive: var(--destructive);
  --color-accent-foreground: var(--accent-foreground);
  --color-accent: var(--accent);
  --color-muted-foreground: var(--muted-foreground);
  --color-muted: var(--muted);
  --color-secondary-foreground: var(--secondary-foreground);
  --color-secondary: var(--secondary);
  --color-primary-foreground: var(--primary-foreground);
  --color-primary: var(--primary);
  --color-popover-foreground: var(--popover-foreground);
  --color-popover: var(--popover);

```

```

--color-card-foreground: var(--card-foreground);
--color-card: var(--card);
--color-foreground: var(--foreground);
--color-background: var(--background);
--radius-sm: calc(var(--radius) * 0.6);
--radius-md: calc(var(--radius) * 0.8);
--radius-lg: var(--radius);
--radius-xl: calc(var(--radius) * 1.4);
--radius-2xl: calc(var(--radius) * 1.8);
--radius-3xl: calc(var(--radius) * 2.2);
--radius-4xl: calc(var(--radius) * 2.6);
}

.dark {
--background: oklch(0.145 0 0);
--foreground: oklch(0.985 0 0);
--card: oklch(0.205 0 0);
--card-foreground: oklch(0.985 0 0);
--popover: oklch(0.205 0 0);
--popover-foreground: oklch(0.985 0 0);
--primary: oklch(0.922 0 0);
--primary-foreground: oklch(0.205 0 0);
--secondary: oklch(0.269 0 0);
--secondary-foreground: oklch(0.985 0 0);
--muted: oklch(0.269 0 0);
--muted-foreground: oklch(0.708 0 0);
--accent: oklch(0.269 0 0);
--accent-foreground: oklch(0.985 0 0);
--destructive: oklch(0.704 0.191 22.216);
--border: oklch(1 0 0 / 10%);
--input: oklch(1 0 0 / 15%);
--ring: oklch(0.556 0 0);
--chart-1: oklch(0.87 0 0);
--chart-2: oklch(0.556 0 0);
--chart-3: oklch(0.439 0 0);
--chart-4: oklch(0.371 0 0);
--chart-5: oklch(0.269 0 0);
--sidebar: oklch(0.205 0 0);
--sidebar-foreground: oklch(0.985 0 0);
--sidebar-primary: oklch(0.488 0.243 264.376);
--sidebar-primary-foreground: oklch(0.985 0 0);
--sidebar-accent: oklch(0.269 0 0);
--sidebar-accent-foreground: oklch(0.985 0 0);
--sidebar-border: oklch(1 0 0 / 10%);
--sidebar-ring: oklch(0.556 0 0);
}

@layer base {
* {
@apply border-border outline-ring/50;
}
body {
@apply bg-background text-foreground;
}
html {
@apply font-mono;
}
}

```

Листинг 3 — routes/__root.tsx (основная оболочка интерфейса)

```
import "../globals.css"
```

```

import {
  Activity,
  AlertCircle,
  CheckCircle2,
  Clock3,
  LayoutGrid,
  MessageCircle,
  ShieldAlert,
  SquareActivity,
} from "lucide-react"
import { Link, Outlet, createRootRoute } from "@tanstack/react-router"
import { useEffect, useState } from "react"

import { Button } from "@components/ui/button"
import { TanStackDevtools } from "@tanstack/react-devtools"
import { TanStackRouterDevtoolsPanel } from "@tanstack/react-router-devtools"
import { cn } from "@lib/utils"

export const Route = createRootRoute({
  component: RootComponent,
})

const menuItems = [
  {
    label: "Обзор",
    icon: LayoutGrid,
    to: "/",
  },
  {
    label: "Мониторинг",
    icon: SquareActivity,
    to: "/monitor",
  },
  {
    label: "Администрирование",
    icon: ShieldAlert,
    to: "/admin",
  },
]

const latestEvents = [
  {
    id: 1,
    title: "Сигнал тревоги обработан",
    detail: "Инцидент INC-002 переведён в активную эскалацию",
    time: "2 мин назад",
    channel: "Оператор",
    severity: "warning" as const,
    icon: AlertCircle,
  },
  {
    id: 2,
    title: "ИИ предложил повторный контакт",
    detail: "Для водителя подготовлен новый звонок по правилу SLA",
    time: "5 мин назад",
    channel: "AI",
    severity: "info" as const,
    icon: Activity,
  },
  {
    id: 3,

```

```

    title: "SMS доставлено клиенту",
    detail: "Сообщение о сдвиге ETA успешно отправлено",
    time: "8 мин назад",
    channel: "SMS",
    severity: "success" as const,
    icon: CheckCircle2,
  },
  {
    id: 4,
    title: "Плановая синхронизация завершена",
    detail: "Обновлены статусы маршрутов и метрики отклонения",
    time: "12 мин назад",
    channel: "Система",
    severity: "info" as const,
    icon: Clock3,
  },
]

function RootComponent() {
  // TODO: extract sidebar
  const [syncTick, setSyncTick] = useState(0)

  useEffect(() => {
    const interval = window.setInterval(() => {
      setSyncTick((value) => value + 1)
    }, 15000)

    return () => window.clearInterval(interval)
  }, [])

  const currentTime = new Date().toLocaleTimeString("ru-RU", {
    hour: "2-digit",
    minute: "2-digit",
  })

  return (
    <>
      <div className="min-h-screen bg-background">
        <div className="mx-auto flex gap-6 p-6">
          <aside className="sticky top-6 flex h-[calc(100vh-3rem)] w-72 shrink-0 flex-col rounded-2xl border bg-slate-950 p-5 text-slate-50">
            <div>
              <p className="text-xs text-slate-400">МКTech // AI-диспетчер</p>
              <h2 className="mt-2 text-2xl font-semibold">Логист-аналитик</h2>
            </div>
            <nav className="mt-6 space-y-1.5">
              {menuItems.map((item) => {
                const Icon = item.icon
                return (
                  <Button
                    key={item.label}
                    variant="ghost"
                    asChild
                    className={cn(
                      "flex w-full justify-start gap-3 rounded-xl px-3 py-6 text-sm",
                      "data-[status=active]:cursor-default data-[status=active]:hover:bg-slate-900",
                      "data-[status=active]:text-slate-50",
                    )}
                  >
                    <Link

```

```

        to={item.to}
        activeOptions={{ exact: true }}
        activeProps={{
          className:
            "bg-slate-900 text-slate-50 active:translate-none!",
        }}
      >
        <Icon className="size-5" />
        {item.label}
      </Link>
    </Button>
  )
  })}
</nav>
<div className="mt-auto rounded-2xl bg-slate-900 p-4">
  <p className="text-xs text-slate-400">Синхронизация</p>
  <p className="text-2xl font-semibold">
    {syncTick % 2 === 0 ? "Онлайн" : "Обновлено"}
  </p>
  <p className="text-sm text-slate-300">{currentTime}</p>
</div>
</aside>

<section className="flex-1 space-y-6">
  <Outlet />
</section>

<aside className="sticky top-6 flex h-[calc(100vh-3rem)] w-72 shrink-0
flex-col rounded-2xl border bg-muted p-5 text-muted-foreground">
  <div className="flex items-center justify-between">
    <div>
      <p className="text-xs">Последние события</p>
    </div>
    <span className="rounded-full bg-background px-2 py-1 text-[11px]
font-medium text-foreground">
      Live
    </span>
  </div>

  <div className="mt-4 space-y-3">
    {latestEvents.map((event) => {
      const Icon = event.icon
      const toneClass =
        event.severity === "warning"
          ? "bg-amber-100 text-amber-700"
          : event.severity === "success"
            ? "bg-emerald-100 text-emerald-700"
            : "bg-slate-100 text-slate-700"

      return (
        <div
          key={event.id}
          className="rounded-xl border border-border/70 bg-background/80
p-3 shadow-sm"
        >
          <div className="flex items-start gap-3">
            <div className={cn("rounded-full p-1.5", toneClass)}>
              <Icon className="size-3.5" />
            </div>
            <div className="min-w-0 flex-1">
              <p className="text-xs font-medium text-foreground line-
clamp-2">

```

```

        {event.title}
      </p>
      <p className="mt-1 text-xs leading-5 text-muted-foreground
line-clamp-2">
        {event.detail}
      </p>
    </div>
  </div>
  <div className="mt-3 flex items-center justify-between text-
[11px] text-muted-foreground">
    <span>{event.time}</span>
    <span>{event.channel}</span>
  </div>
</div>
)
  })}
</div>
</aside>
</div>
</div>
<TanStackDevtools
  config={{
    position: "bottom-right",
  }}
  plugins={[
    {
      name: "TanStack Router",
      render: <TanStackRouterDevtoolsPanel />,
    },
  ]}
/>
</>
)
}

```

Листинг 4 — Таблица «Рейсы» на странице «Обзор»

```

<Card>
  <CardHeader className="flex flex-col gap-4 md:flex-row md:items-end
md:justify-between">
    <div>
      <p className="text-xs text-muted-foreground">
        Закрепленные за вами рейсы
      </p>
      <CardTitle className="text-xl">Активные рейсы</CardTitle>
    </div>
    <div className="flex flex-col gap-2 md:flex-row md:items-center">
      <Input
        placeholder="Поиск по номеру заявки, маршруту или водителю"
        value={search}
        onChange={(event) => setSearch(event.target.value)}
        className="w-full md:w-128"
      />
      <Select value={statusFilter} onChange={setStatusFilter}>
        <SelectTrigger className="w-full md:w-52">
          <SelectValue placeholder="Все статусы" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="all">Все статусы</SelectItem>
          {tripStatusOptions.map((option) => (
            <SelectItem key={option.value} value={option.value}>
              {option.label}
            </SelectItem>
          ))}
        </SelectContent>
      </div>
    </div>
  </CardHeader>
  <Table>
    <TableHeader>
      <TableHead>
        <TableTr>
          <TableTh>Статус</TableTh>
          <TableTh>Маршрут</TableTh>
          <TableTh>Водитель</TableTh>
          <TableTh>Номер заявки</TableTh>
        </TableTr>
      </TableHead>
      <TableBody>
        {trips.map((trip) => (
          <TableTr>
            <TableTd>{trip.status}</TableTd>
            <TableTd>{trip.route}</TableTd>
            <TableTd>{trip.driver}</TableTd>
            <TableTd>{trip.number}</TableTd>
          </TableTr>
        ))}
      </TableBody>
    </TableHeader>
  </Table>
</Card>

```

```

        </SelectedItem>
      )}}
    </SelectContent>
  </Select>
</div>
</CardHeader>
<CardContent className="space-y-4">
  <div className="overflow-hidden rounded-xl border">
    <Table className="table-fixed">
      <TableHeader>
        <TableRow>
          <TableHead>
            <button
              className="flex items-center gap-1 font-medium cursor-pointer
bg-accent rounded-xl px-2 py-1"
              onClick={() => toggleSort("route")}
            >
              <span>Номер заявки</span>
              {sortKey === "route" ? (
                sortDir === "asc" ? (
                  <ArrowUpWideNarrow className="size-5" />
                ) : (
                  <ArrowDownWideNarrow className="size-5" />
                )
              ) : null}
            </button>
          </TableHead>
          <TableHead>
            <button
              className="flex items-center gap-1 font-medium cursor-pointer
bg-accent rounded-xl px-2 py-1"
              onClick={() => toggleSort("status")}
            >
              <span>Статус</span>
              {sortKey === "status" ? (
                sortDir === "asc" ? (
                  <ArrowUpWideNarrow className="size-5" />
                ) : (
                  <ArrowDownWideNarrow className="size-5" />
                )
              ) : null}
            </button>
          </TableHead>
          <TableHead>Транспорт</TableHead>
          <TableHead>
            <button
              className="flex items-center gap-1 font-medium cursor-pointer
bg-accent rounded-xl px-2 py-1"
              onClick={() => toggleSort("eta")}
            >
              <span>ETA</span>
              {sortKey === "eta" ? (
                sortDir === "asc" ? (
                  <ArrowUpWideNarrow className="size-5" />
                ) : (
                  <ArrowDownWideNarrow className="size-5" />
                )
              ) : null}
            </button>
          </TableHead>
          <TableHead>Риск нарушения SLA</TableHead>
          <TableHead>Обновлено</TableHead>

```

```

    </TableRow>
  </TableHeader>
  <TableBody>
    {isLoading ? (
      <TableRow>
        <TableCell
          colSpan={6}
          className="py-8 text-center text-muted-foreground"
        >
          Загрузка закреплённых рейсов...
        </TableCell>
      </TableRow>
    ) : visibleRoutes.length === 0 ? (
      <TableRow>
        <TableCell
          colSpan={6}
          className="py-8 text-center text-muted-foreground"
        >
          Подходящих рейсов не найдено.
        </TableCell>
      </TableRow>
    ) : (
      visibleRoutes.map((route) => (
        <TableRow
          key={route.id}
          className="cursor-pointer"
          onClick={() => openRouteDetails(route)}
        >
          <TableCell>
            <div className="space-y-1">
              <p className="font-medium">{route.route}</p>
              <p className="text-xs text-muted-foreground">
                {route.id}
              </p>
            </div>
          </TableCell>
          <TableCell>
            <Badge
              className={getTripStatusClassName(route.status)}
              variant="secondary"
            >
              {route.status}
            </Badge>
          </TableCell>
          <TableCell>
            <div className="space-y-1">
              <p className="font-medium">{route.vehicle}</p>
              <p className="text-xs text-muted-foreground">
                {route.driver}
              </p>
            </div>
          </TableCell>
          <TableCell>{route.eta}</TableCell>
          <TableCell>{route.sla}</TableCell>
          <TableCell>{route.updatedAt}</TableCell>
        </TableRow>
      ))
    )}
  </TableBody>
</Table>
</div>
<div className="flex items-center justify-between">

```

```

    <p className="text-sm text-muted-foreground">
      Показано {visibleRoutes.length} из {filteredRoutes.length} рейсов
    </p>
    <div className="flex items-center gap-2">
      <Button
        variant="outline"
        size="sm"
        disabled={safePage === 1}
        onClick={() => setPage((current) => Math.max(1, current - 1))}
      >
        Назад
      </Button>
      <Button
        size="sm"
        disabled={safePage === pageCount}
        onClick={() =>
          setPage((current) => Math.min(pageCount, current + 1))
        }
      >
        Далее
      </Button>
    </div>
  </div>
</CardContent>
</Card>

```

Листинг 5 — components/timeline.tsx (компонент для отображения элементов в заданном порядке вдоль стрелки)

```

import * as React from "react"

import { Badge } from "@components/ui/badge"
import { cn } from "@lib/utils"

const Timeline = React.forwardRef<
  HTMLDivElement,
  React.HTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div ref={ref} className={className} {...props} />
))
Timeline.displayName = "Timeline"

const TimelineItem = React.forwardRef<
  HTMLDivElement,
  React.LiHTMLAttributes<HTMLDivElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("group relative pb-2 pl-8 sm:pl-[7em]", className)}
    {...props}
  />
))
TimelineItem.displayName = "TimelineItem"

const TimelineHeader = React.forwardRef<
  HTMLParagraphElement,
  React.HTMLAttributes<HTMLParagraphElement>
>(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn(

```

```

        "mb-1 flex flex-col items-start before:absolute before:left-2 before:h-full
before:-translate-x-1/2",
        "before:translate-y-3 before:self-start before:bg-slate-300 before:px-px
after:absolute",
        "after:left-2 after:box-content after:h-2 after:w-2 after:-translate-x-1/2",
        "after:translate-y-1.5 after:rounded-full after:border-4 after:border-primary-
foreground/95",
        "after:bg-foreground group-last:before:hidden sm:flex-row sm:before:left-0
sm:before:ml-[5em]",
        "sm:after:left-0 sm:after:ml-[5em]",
        className,
      )}
      {...props}
    />
  ))
  TimelineHeader.displayName = "TimelineHeader"

  const TimelineTitle = React.forwardRef<
    HTMLDivElement,
    React.HTMLAttributes<HTMLDivElement>
  >(({ className, children, ...props }, ref) => (
    <div ref={ref} className={cn("text-base text-primary", className)} {...props}>
      {children}
    </div>
  ))
  TimelineTitle.displayName = "TimelineTitle"

  const TimelineTime = ({
    className,
    variant = "default",
    ...props
  }: React.ComponentProps<typeof Badge>) => {
    return (
      <Badge
        className={cn(
          "left-0 mb-3 inline-flex h-6 translate-y-0.5 items-center justify-center
text-xs uppercase sm:absolute sm:mb-0",
          className,
        )}
        variant={variant}
        {...props}
      >
        {props.children}
      </Badge>
    )
  }
  TimelineTime.displayName = "TimelineTime"

  const TimelineDescription = React.forwardRef<
    HTMLDivElement,
    React.HTMLAttributes<HTMLDivElement>
  >(({ className, ...props }, ref) => (
    <div
      ref={ref}
      className={cn("text-muted-foreground text-sm", className)}
      {...props}
    />
  ))
  TimelineDescription.displayName = "TimelineDescription"

  export {
    Timeline,

```

```
TimelineItem,  
TimelineHeader,  
TimelineTime,  
TimelineTitle,  
TimelineDescription,  
}
```

Отзыв о производственной практике

Обучающийся Томчук Григорий Сергеевич проходил производственную технологическую
(инициалы, фамилия)
(проектно-технологическую) практику в организации ООО «СОЛИНГ» с 29.06.2026 по
26.07.2026.

В течение прохождения практики обучающийся Томчук Григорий Сергеевич
(инициалы, фамилия)

Внимательно относился к выполняемой работе, координировал разработку с коллегами по команде.

(внимательно относился к выполняемой работе, помогал сотрудникам с расчетами различных показателей и т.д.).

Поручаемую работу, обучающийся Г.С. Томчук выполнял добросовестно и
аккуратно.

(добросовестно, аккуратно и т.д.)

Показал себя дисциплинированным и ответственным работником

(дисциплинированным, ответственным, исполнительным, пунктуальным работником)

Замечания о прохождении практики: отсутствуют

В целом работу обучающегося Томчука Григория Сергеевича можно оценить
на отлично

(отлично, хорошо, удовлетворительно, неудовлетворительно)

Подпись руководителя практики от профильной организации

ген. директор

(должность, уч. степень, звание)



М.П.

Д.А. Яснов

(инициалы, фамилия)